

Suppose that disk hardware allows you to choose the size of a disk block arbitrarily, but that the time it takes to read the disk block is $a+bt$, where a and b are specified constants and t is the minimum degree for a B-tree using blocks of the selected size. Describe how to choose t so as to minimize (approximately) the B-tree search time. Suggest an optimal value of t for the case in which $a = 5$ milliseconds and $b = 10$ microseconds.

18.3 Deleting a key from a B-tree

Deletion from a B-tree is analogous to insertion but a little more complicated, because you can delete a key from any node—not just a leaf—and when you delete a key from an internal node, you must rearrange the node’s children. As in insertion, you must guard against deletion producing a tree whose structure violates the B-tree properties. Just as a node should not get too big due to insertion, a node must not get too small during deletion (except that the root is allowed to have fewer than the minimum number $t - 1$ of keys). And just as a simple insertion algorithm might have to back up if a node on the path to where the key is to be inserted is full, a simple approach to deletion might have to back up if a node (other than the root) along the path to where the key is to be deleted has the minimum number of keys.

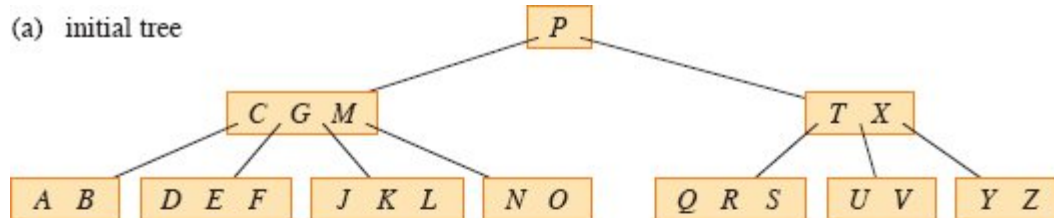
The procedure B-TREE-DELETE deletes the key k from the subtree rooted at x . Unlike the procedures TREE-DELETE on page 325 and RB-DELETE on page 348, which are given the node to delete—presumably as the result of a prior search—B-TREE-DELETE combines the search for key k with the deletion process. Why do we combine search and deletion in B-TREE-DELETE? Just as B-TREE-INSERT prevents any node from becoming overfull (having more than $2t - 1$ keys) while making a single pass down the tree, B-TREE-DELETE prevents any node from becoming underfull (having fewer than $t - 1$ keys) while also making a single pass down the tree, searching for and ultimately deleting the key.

To prevent any node from becoming underfull, the design of B-TREE-DELETE guarantees that whenever it calls itself recursively on a

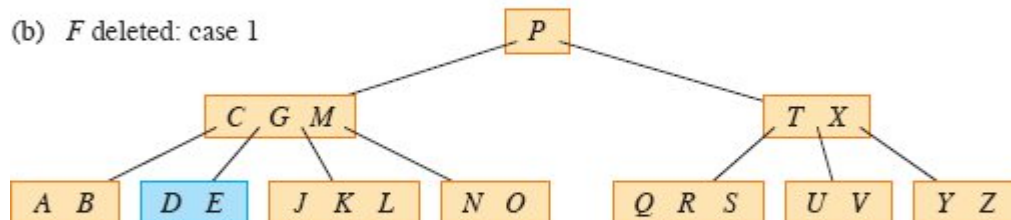
node x , the number of keys in x is at least the minimum degree t at the time of the call. (Although the root may have fewer than t keys and a recursive call may be made *from* the root, no recursive call is made *on* the root.) This condition requires one more key than the minimum required by the usual B-tree conditions, and so a key might have to be moved from x into one of its child nodes (still leaving x with at least the minimum $t - 1$ keys) before a recursive call is made on that child, thus allowing deletion to occur in one downward pass without having to traverse back up the tree.

We describe how the procedure B-TREE-DELETE(T, k) deletes a key k from a B-tree T instead of presenting detailed pseudocode. We examine three cases, illustrated in Figure 18.8. The cases are for when the search arrives at a leaf, at an internal node containing key k , and at an internal node not containing key k . As mentioned above, in all three cases node x has at least t keys (with the possible exception of when x is the root). Cases 2 and 3—when x is an internal node—guarantee this property as the recursion descends through the B-tree.

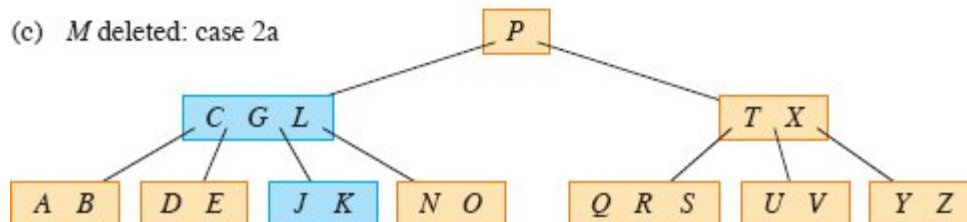
(a) initial tree



(b) F deleted: case 1



(c) M deleted: case 2a



(a) initial tree

D

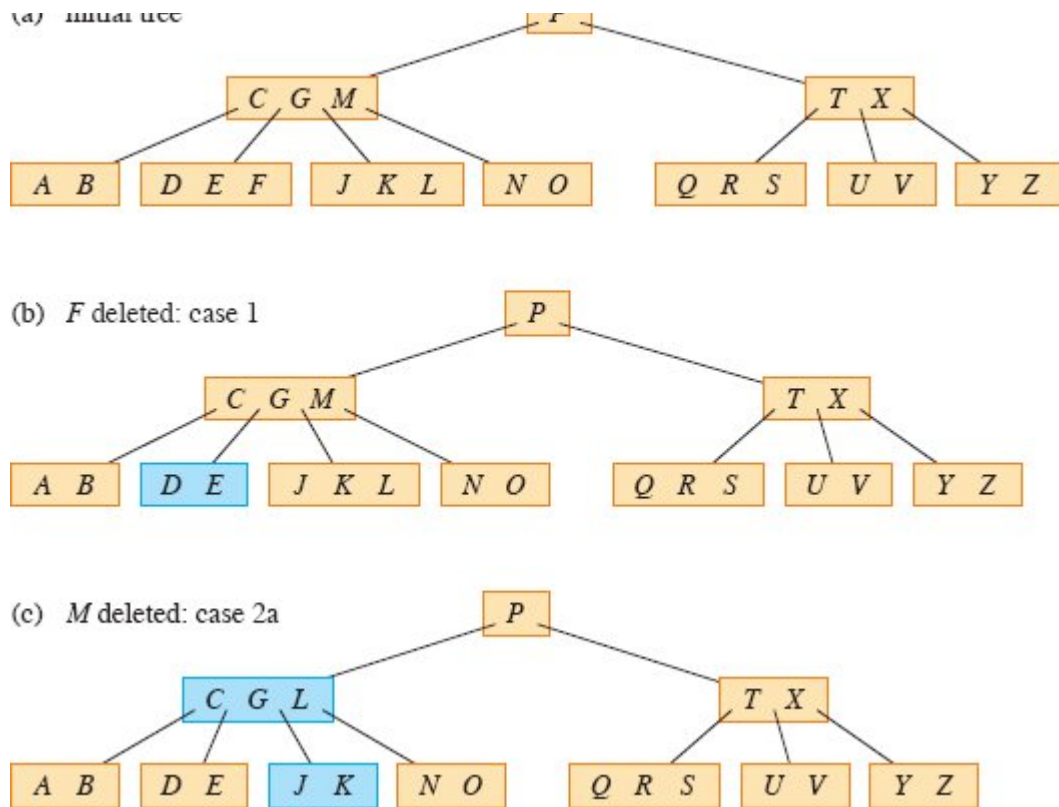


Figure 18.8 Deleting keys from a B-tree. The minimum degree for this B-tree is $t = 3$, so that, other than the root, every node must have at least 2 keys. Blue nodes are those that are modified by the deletion process. **(a)** The B-tree of Figure 18.7(e). **(b)** Deletion of F , which is case 1: simple deletion from a leaf when all nodes visited during the search (other than the root) have at least $t = 3$ keys. **(c)** Deletion of M , which is case 2a: the predecessor L of M moves up to take M 's position. **(d)** Deletion of G , which is case 2c: push G down to make node $DEGJK$ and then delete G from this leaf (case 1). **(e)** Deletion of D , which is case 3b: since the recursion cannot descend to node CL because it has only 2 keys, push P down and merge it with CL and TX to form $CLP TX$. Then delete D from a leaf (case 1). **(e⁰)** After (e), delete the empty root. The tree shrinks in height by 1. **(f)** Deletion of B , which is case 3a: C moves to fill B 's position and E moves to fill C 's position.

Case 1: The search arrives at a leaf node x . If x contains key k , then delete k from x . If x does not contain key k , then k was not in the B-tree and nothing else needs to be done.

Case 2: The search arrives at an internal node x that contains key k . Let $k = x.key_j$. One of the following three cases applies, depending on the number of keys in $x.c_j$ (the child of x that precedes k) and $x.c_{j+1}$ (the child of x that follows k).

Case 2a: $x.c_i$ has at least t keys. Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x . (Key k' can be found and deleted in a single downward pass.)

Case 2b: $x.c_i$ has $t - 1$ keys and $x.c_{i+1}$ has at least t keys. This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x . (Again, finding and deleting k' can be done in a single downward pass.)

Case 2c: Both $x.c_i$ and $x.c_{i+1}$ have $t - 1$ keys. Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t - 1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

Case 3: The search arrives at an internal node x that does not contain key k . Continue searching down the tree while ensuring that each node visited has at least t keys. To do so, determine the root $x.c_i$ of the appropriate subtree that must contain k , if k is in the tree at all. If $x.c_i$ has only $t - 1$ keys, execute case 3a or 3b as necessary to guarantee descending to a node containing at least t keys. Then finish by recursing on the appropriate child of x .

Case 3a: $x.c_i$ has only $t - 1$ keys but has an immediate sibling with at least t keys. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x , and moving the appropriate child pointer from the sibling into $x.c_i$.

Case 3b: $x.c_i$ and each of $x.c_i$'s immediate siblings have $t - 1$ keys. (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

In cases 2c and 3b, if node x is the root, it could end up having no keys. When this situation occurs, then x is deleted, and x 's only child $x.c_1$ becomes the new root of the tree. This action decreases the height

of the tree by one and preserves the property that the root of the tree contains at least one key (unless the tree is empty).

Since most of the keys in a B-tree are in the leaves, deletion operations often end up deleting keys from leaves. The B-TREE-DELETE procedure then acts in one downward pass through the tree, without having to back up. When deleting a key in an internal node x , however, the procedure might make a downward pass through the tree to find the key's predecessor or successor and then return to node x to replace the key with its predecessor or successor (cases 2a and 2b). Returning to node x does not require a traversal through all the levels between x and the node containing the predecessor or successor, however, since the procedure can just keep a pointer to x and the key position within x and put the predecessor or successor key directly there.

Although this procedure seems complicated, it involves only $O(h)$ disk operations for a B-tree of height h , since only $O(1)$ calls to DISK-READ and DISK-WRITE are made between recursive invocations of the procedure. The CPU time required is $O(th) = O(t \log_t n)$.

Exercises

18.3-1

Show the results of deleting C , P , and V , in order, from the tree of Figure 18.8(f).

18.3-2

Write pseudocode for B-TREE-DELETE.

Problems

18-1 *Stacks on secondary storage*

Consider implementing a stack in a computer that has a relatively small amount of fast primary memory and a relatively large amount of slower disk storage. The operations PUSH and POP work on single-word